

Universidade Federal do Rio Grande do Norte
Departamento de Informática e Matemática Aplicada
DIM0548 – Engenharia de Linguagens

DOCUMENTAÇÃO - ANALISADOR SINTÁTICO

KOJITO

EXPEDITO HEBERT FIRMINO DA ROCHA
FRANCISCO GABRIEL COSTA BESSA
JOSE CARLOS DA SILVA NASCIMENTO
PEDRO VINÍCIUS BARBOSA PEREIRA
SABRINA DA SILVA BARBOSA VENCESLAU

Natal - RN
2026

SUMÁRIO

1. ESTRUTURA DOS PROGRAMAS DA LINGUAGEM	3
1.1 Código-fonte mínimo compilável	3
1.2 Exemplos gerais	4
1.2.1 Subprogramas	4
1.2.1.0 Função	4
1.2.1.1 Função Pura	4
1.2.1.2 Procedimento	5
1.2.2 Atribuição	5
1.2.3 Estruturas de Repetição	5
1.2.3.0 While	5
1.2.3.1 Loop	6
1.2.3.2 For	6
1.2.4 Estruturas de Decisão	6
1.2.4.0 If-Else	6
1.2.4.1 Match	6
2. ESTRUTURA DOS ANALISADORES	7
2.1 Analisador Léxico: principais tarefas, decisões e dificuldades	7
2.2 Analisador Sintático: principais tarefas, decisões e dificuldades	8
2.3 Limitações da implementação atual	9
3. USO DO ANALISADOR SINTÁTICO	9
3.1 Como gerar o analisador sintático	9
3.2 Programas de Teste e resultados esperados	10

1. ESTRUTURA DOS PROGRAMAS DA LINGUAGEM

A estrutura de um programa em uma linguagem de programação imperativa e compilada se dá a partir de como as coisas são postas em código, ou seja, como os comandos são feitos para que se possa gerar um executável. Dessa forma, tratando-se de Kojito, mostraremos como um programa será estruturado em nossa linguagem nas próximas seções.

1.1 CÓDIGO-FONTE MÍNIMO COMPILÁVEL

Utilizamos o algoritmo do Merge Sort estruturado em nossa linguagem, como exemplo de um código-fonte compilável:

```
func merge(left : &[u_int32], right : &[u_int32]) -> Vec<u_int32> {
    let result: Vec<u_int32> = Vec::with_capacity(left.len() + right.len());

    let r : [u_int32; 5] = left;
    let i : u_int32 = 0;
    let j : u_int32 = 0;
    while (i < left.len() && j < right.len()) {
        if (left[i] <= right[j]) {
            result.push(left[i]);
            i++;
        } else {
            result.push(right[j]);
            j++;
        }
    }
    while (i < left.len()) {
        result.push(left[i]);
        i++;
    }
    while (j < right.len()) {
        result.push(right[j]);
        j++;
    }

    return result;
}

func merge_sort(arr: &[u_int32]) -> Vec<u_int32> {
    if (arr.len() <= 1) {
        return arr.to_vec();
    }
    let mid : u_size = arr.len() / 2;
    let left : u_int32 = merge_sort(&arr[..mid]);
    let right : u_int32 = merge_sort(&arr[mid..]);
```

```
    return merge(&left, &right);
}

func foo() -> s_int32 {
    return s;
}

func main(i:u_int8) -> Result<(), String> {
    let array : [s_int32; 5] = [5, 4, 3, 2, 1];
    let result : Vec<u_int32> = merge_sort(&array);
    for (i in result) {
        print("{} ", i);
    }
    print("\n");
    return Ok(());
}
```

1.2 EXEMPLOS GERAIS

Para entender melhor como nossa linguagem é estruturada, abaixo seguem os exemplos gerais de como escrever funções, declarações, estruturas de decisão, de repetição, structs e qualquer outro componente possível de ser escrito em nossa linguagem:

1.2.1 SUBPROGRAMAS

Nossa linguagem terá três categorias de subprogramas: função, função pura e procedimento.

1.2.1.0 FUNÇÃO

As funções em nossa linguagem terão um tipo de entrada e, obrigatoriamente, um tipo de retorno, elas podem causar efeitos colaterais, portanto podem alterar variáveis recebidas por referência e variáveis globais. Uma função terá a seguinte sintaxe:

```
func noFunc(u8 : u_int8, u16 : u_int16) -> char {
    u8 = 3;
    return 'c';
}
```

1.2.1.1 FUNÇÃO PURA

As funções puras terão um tipo de entrada e, obrigatoriamente, um tipo de retorno, elas não podem causar efeitos colaterais, portanto o programador terá que seguir um série de regras ao escrever dentro delas, como:

```
pure func pureNoFunc(u8 : u_int8, u16 : u_int16) -> char {  
    return 'f';  
}
```

1.2.1.2 PROCEDIMENTO

Os procedimentos terão apenas tipos de entrada, também podem não possuir nenhum, eles podem causar efeitos colaterais, portanto podem alterar variáveis recebidas por referência e variáveis globais. Os procedimentos terão a seguinte sintaxe:

```
proced procedures(u8 : u_int8) {  
    u8 = 3;  
}
```

1.2.2 ATRIBUIÇÃO

Considerando que temos como um de nossos objetivos a segurança de memória, previsibilidade e tipagem forte — Kojito adotará um conjunto restrito e controlado de formas de atribuição.

Dito isto, será suportada a atribuição simples, sendo esta a forma principal de modificação de estado, seguindo uma sintaxe clara e obrigatoriamente tipada,

```
let s8 : s_int8 = 8;
```

1.2.3 ESTRUTURAS DE REPETIÇÃO

Nossa linguagem terá apenas as estruturas while, loop e for. Optamos por não incluir o do-while, em alternativa temos o loop que é mais simples, tornando a linguagem mais fácil de aprender e mais segura (evitando execuções acidentais de blocos de código), além de que o loop tem mais possibilidade de uso do que o do-while, que é geralmente pouco utilizado.

1.2.3.0 WHILE

A estrutura do while é feita da seguinte forma:

```
while (true) {}
```

1.2.3.1 LOOP

A estrutura do Loop é feita da seguinte forma:

```
loop {  
    break;  
}
```

1.2.3.2 FOR

A estrutura do For é feita da seguinte forma:

```
for (i in 0..9) {  
    continue;  
}
```

1.2.4 ESTRUTURAS DE DECISÃO

Nossa linguagem terá apenas as estruturas if-else e match:

```
for (i in 0..9) {  
    continue;  
}
```

1.2.4.0 IF-ELSE

A estrutura do if-else é feita da seguinte forma:

```
if (true) {}
```

1.2.4.1 MATCH

A estrutura do match é feita da seguinte forma:

```
match (res) {  
    Ok(u) = {};  
    Erro(s) = {};  
}
```

Os demais exemplos podem ser consultados no arquivo `examples/all_tests.kjt`.

2. ESTRUTURA DOS ANALISADORES

O compilador da linguagem Kojito foi desenvolvido utilizando duas ferramentas amplamente empregadas na construção de compiladores: o Flex, responsável pela análise léxica, e o Bison, responsável pela análise sintática. Essa divisão permite separar o reconhecimento dos elementos básicos da linguagem da validação de sua estrutura gramatical, tornando o projeto mais organizado e modular.

Durante a compilação, o código-fonte é inicialmente processado pelo analisador léxico, que identifica palavras reservadas, operadores, identificadores, literais e demais símbolos definidos pela linguagem. Cada elemento reconhecido é convertido em um token, que posteriormente é encaminhado ao analisador sintático.

O analisador sintático recebe a sequência de tokens produzida pelo analisador léxico e verifica se ela está de acordo com as regras gramaticais da linguagem Kojito. Para isso, foi definida uma gramática livre de contexto capaz de reconhecer construções como declarações de variáveis, funções, procedimentos, estruturas de decisão, estruturas de repetição, expressões, tipos compostos e demais componentes da linguagem.

A comunicação entre os dois analisadores ocorre de forma transparente: o Flex identifica os tokens e o Bison os consome para validar a estrutura do programa. Caso seja encontrada alguma inconsistência sintática, o analisador informa a linha e o trecho do código onde o erro ocorreu, facilitando sua correção.

A arquitetura adotada segue o fluxo tradicional de compilação, representado da seguinte forma:

Código-fonte → Analisador Léxico (Flex) → Tokens → Analisador Sintático (Bison) → Programa Validado

Essa abordagem simplifica a manutenção do compilador, facilita a evolução da linguagem e permite que novas funcionalidades sejam adicionadas futuramente sem a necessidade de reestruturar completamente o processo de análise.

2.1 ANALISADOR LÉXICO: PRINCIPAIS TAREFAS, DECISÕES E DIFICULDADES

O analisador léxico da linguagem Kojito foi implementado utilizando o Flex e tem como principal objetivo transformar o código-fonte em uma sequência de tokens compreensíveis pelo analisador sintático. Para isso, foram definidas expressões regulares capazes de reconhecer palavras reservadas, identificadores, operadores, delimitadores, tipos primitivos e literais da linguagem.

Entre os elementos reconhecidos encontram-se palavras-chave como `func`, `let`, `if`, `while`, `struct` e `enum`, além de tipos como `u_int32`, `float64`, `bool` e estruturas genéricas como `Vec`, `Result` e `Option`.

Também foram implementadas regras para ignorar comentários de linha e comentários de bloco, evitando que esses elementos interfiram na análise sintática. Espaços em branco e quebras de linha são tratados adequadamente, sendo utilizadas apenas para controle de posição e geração de mensagens de erro.

Uma das principais dificuldades encontradas durante o desenvolvimento foi garantir que operadores compostos fossem reconhecidos corretamente sem conflito com operadores simples. Exemplos incluem os pares `>=` e `>`, `==` e `=`, além dos operadores de atribuição composta, como `+=` e `*=`.

2.2 ANALISADOR SINTÁTICO: PRINCIPAIS TAREFAS, DECISÕES E DIFICULDADES

O analisador sintático foi desenvolvido utilizando o Bison e é responsável por validar a estrutura dos programas escritos em Kojito. Sua implementação baseia-se em uma gramática livre de contexto capaz de reconhecer os diferentes elementos da linguagem e verificar se eles foram utilizados de forma correta.

Dito isso, uma dificuldade que enfrentamos foi a de decidir como deveria ser a definição do símbolo de partida (Program), pois estávamos na dúvida de como deveria ser visto o programa e como, a partir dele seriam desencadeadas as demais partes que compõem um programa na nossa linguagem. Todavia, apesar dessa dificuldade, optamos por definir o programa como um conjunto de subprogramas, declarações de variáveis globais e uma main.

Outra dificuldade que enfrentamos foi a de definir de maneira eficiente a estrutura sintática para a chamada de Subprogramas, representado pelo SubprogramCall. Isso se deve ao fato de ter várias maneiras de chamar um subprograma, como por exemplo:

```
SubprogramCall: ID MaybeParams '.' SubprogramCall {}  
               | ID '.' SubprogramCall {} // foo.poo()  
               | ID MaybeParams {}  
               ;
```

```
foo();  
foo(1, 2);  
foo().boo();
```

Como pode ter uma função sem parâmetros, foi criado a estrutura sintática: MaybeParams, que inclui tanto a chamada de função sem parâmetros quanto a chamada com parâmetros que ocorre de maneira recursiva através da operação ParamsToCall.

Dito isto, estas foram as principais dificuldades que enfrentamos nessa etapa do projeto. Porém, apesar de termos evidenciado apenas essas duas situações que consideramos as mais difíceis de elaborarmos suas estruturas sintáticas, nosso analisador sintático comporta outras regras de como deve ser estruturada a função principal (main), o escopo, as declarações, as atribuições, arrays, estruturas de

decisão e de repetições, entre outras que podem ser localizadas no arquivo `src/parser.y` do projeto. Além disso, também incluímos uma pequena coleção de tipos que são aceitos em nossa linguagem.

Por fim, apesar desses desafios fizemos o possível para manter nossa linguagem funcionando e dentro do contexto ao qual nos propomos a fazê-la, de modo que possa auxiliar no aprendizado da programação, especialmente para aqueles que já tiveram algum contato inicial com programação, mas ainda não têm um conhecimento robusto na área.

2.3 LIMITAÇÕES DA IMPLEMENTAÇÃO ATUAL

Até o momento ainda não pensamos como iremos incluir a estrutura sintática para o enum, tornando um aspecto limitador da implementação atual. Porém, estamos nos empenhando para que essa estrutura se concretize o mais rápido possível, pois estamos cientes que para a etapa seguinte (analisador semântico), todos os tokens precisam estar corretamente definidos, para que haja o bom funcionamento do analisador semântico.

3. USO DO ANALISADOR SINTÁTICO

Para utilizar o analisador sintático definimos o seguinte comando:

```
./compilador < examples/all_tests.kjt
```

De modo que, o `./compilador` é o binário gerado capaz de ler arquivos e o `all_tests.kjt` é o arquivo que contém todos os testes que pensamos como experimento para saber se, de fato, todos os símbolos estão sendo devidamente identificados, bem como se cada token está ocupando seu devido lugar, ou seja, se a estrutura do programa escrito está casando com a estrutura sintática definida, além de identificar erros quando a escrita do programa não respeita a estrutura sintática estabelecida para a linguagem.

3.1 COMO GERAR O ANALISADOR SINTÁTICO.

Para gerar o analisador sintático é necessário realizar os seguintes comandos:

```
# 1. flex lexer.l
# 2. yacc -d -v parser.y
# 3. gcc y.tab.c lex.yy.c -o compilador
```

Assim, no primeiro comando é gerado um arquivo `lex.yy.c` contendo um programa em C a partir das especificações léxicas que definimos para a nossa linguagem, de modo que para cada token a ser analisado lexicalmente, a especificação contém uma expressão regular e uma ação.

No segundo comando é gerado um arquivo `y.tab.c` contendo outro programa em C onde os tokens fornecidos pelo analisador léxico estarão agrupados em estruturas sintáticas, de modo que seja possível construir uma árvore sintática correspondente a essas estruturas. Salientando que as flags `-d` e `-v` servem

para gerar um cabeçalho de tokens e para gerar um relatório de estados, respectivamente, sendo uma boa prática para avaliar se o que foi gerado corresponde ao que se espera que seja.

Por fim, no terceiro comando é gerado um arquivo binário do qual será possível encontrar a conexão entre os programas do lexico e do parser, sendo possível passar um programa na linguagem kojito para ser analisado em cima desse binário gerado. Cabe destacar ainda que a flag -o serve para nomear o arquivo de saída a.out que, no exemplo de comando utilizado, optamos por nomear de compilador, fazendo jus ao que ocorre nesta etapa.

3.2 PROGRAMAS DE TESTE E RESULTADOS ESPERADOS

Os programas de teste podem ser encontrados seguindo o caminho src/examples. Nesse diretório terão quatro arquivos de testes com programas escritos na nossa linguagem (Kojito). Assim, para executar os testes basta dar o seguinte comando:

```
./compilador < examples/[nome_do_arquivo_de_teste]
```

Ao realizar esse comando, é esperado que tudo o que está escrito nesses arquivos seja devidamente identificado, não havendo erros de símbolos, nem de estrutura sintática, de modo que, caso seja utilizado algum símbolo ou estrutura que não faz parte do universo da nossa linguagem seja devidamente detectado e indicado o problema lexical ou sintático.